

负载均衡-概念讲解

1. 先给结论

负载均衡不是“把请求平均分给几台服务器”这么简单。

一句话理解：

负载均衡 = 把请求、连接、任务或数据访问合理分配到多个处理节点，避免单点过载，提升系统性能、可用性、扩展性和故障隔离能力。

更考试化的表达：

负载均衡通过一定的调度策略，将客户端请求分发到后端服务器集群中的合适节点，使多个节点共同承担业务压力，从而提高系统吞吐量、并发处理能力、可用性和横向扩展能力。

最重要的理解：

负载均衡的本质不是平均，而是合理。

合理的意思是：

- 请求能分出去。
- 节点不能过载。
- 异常节点要被摘除。
- 有状态请求不能乱转发。
- 热点数据不能全部压到一个节点。
- 扩容后流量能自动进入新节点。
- 故障时能快速切换。

2. 为什么需要负载均衡

最简单的网站或服务可能只有一台应用服务器：

客户端 -> 应用服务器 -> 数据库

这种结构的问题是：

- 并发能力有限。
- 单台服务器 CPU、内存、连接数有限。
- 机器故障后服务整体不可用。
- 高峰流量来了只能更换更强机器，成本高且有上限。

当业务增长后，通常会部署多台应用服务器：

客户端 -> 负载均衡器 -> 应用服务器1
-> 应用服务器2
-> 应用服务器3

负载均衡器负责把请求分发到后端多个实例。

这样带来四个核心价值：

- 性能：多个节点共同处理请求，提高吞吐量。
- 可用性：单个节点故障时可以摘除，避免整体不可用。
- 扩展性：增加节点即可提升整体处理能力。
- 维护性：可以灰度、下线、升级部分节点。

教材中的大型网站架构演进也体现了这个过程：

- 单体架构
 - > 应用和数据分离
 - > 使用缓存
 - > 使用服务集群
 - > 通过负载均衡调度服务器分发请求
 - > 数据库读写分离
 - > 反向代理和 CDN
 - > 分布式文件系统、分布式数据库

3. 负载均衡解决的不是一个问题

很多同学会把负载均衡只记成“分担压力”。这样不够。

软考中它常和这些质量属性绑定出现：

- | 质量属性 | 负载均衡的作用 |
- | --- |
- | 性能 | 分散请求，提高吞吐量，降低单点压力 |
- | 可用性 | 结合健康检查，摘除故障节点 |
- | 可扩展性 | 增加实例后自动分担流量 |
- | 可靠性 | 避免单节点异常导致整体失败 |
- | 可维护性 | 支持灰度发布、滚动升级、临时下线 |
- | 安全性 | 可作为统一入口，配合限流、访问控制和防护 |

考试表达可以这样写：

- 负载均衡通过服务集群和请求分发机制，避免单点过载和单点故障。
- 结合健康检查、故障摘除、动态权重和灰度发布，可以提高系统的吞吐量、可用性和可扩展性。

4. 负载均衡的基本组成

4.1 客户端

发起请求的一方。

例如：

- 浏览器。
- 手机 App。
- Unity/Cocos 游戏客户端。
- 第三方支付回调。
- 内部服务调用方。

4.2 负载均衡器

负责接收请求并选择后端节点。

常见形态：

- 硬件负载均衡设备。
- 云厂商 SLB / ALB / CLB。
- LVS。
- Nginx。
- HAProxy。
- API 网关。
- Kubernetes Service / Ingress。
- 服务注册中心配合客户端负载均衡。

4.3 后端服务实例

真正处理请求的节点。

例如：

```
登录服务实例1
登录服务实例2
登录服务实例3
```

4.4 健康检查

负载均衡器要知道哪些节点可用。

健康检查可以检查：

- 端口是否可连接。
- HTTP 接口是否返回 200。
- 应用是否正常。
- 响应时间是否超限。
- 错误率是否过高。
- CPU、内存、队列积压是否异常。

没有健康检查的负载均衡是不完整的。

5. 负载均衡常见位置

5.1 DNS 负载均衡

通过 DNS 把同一个域名解析到多个 IP。

例如：

```
api.game.com -> 机房A IP
api.game.com -> 机房B IP
api.game.com -> 机房C IP
```

优点：

- 实现简单。
- 适合地域调度。
- 可用于多机房流量分配。

缺点：

- DNS 缓存导致切换不够实时。
- 调度粒度较粗。
- 健康检查和故障切换能力相对有限。

适合：

全局流量调度
多地域访问
就近接入

5.2 四层负载均衡

四层指传输层，主要基于 IP、端口和 TCP/UDP 连接转发。

典型技术：

LVS
云厂商 CLB / SLB 的四层模式
部分 HAProxy 模式

特点：

- 性能高。
- 转发开销低。
- 不关心 HTTP URL、Header、Cookie。
- 适合高并发 TCP/UDP 转发。

它能根据：

源 IP
目标 IP
源端口
目标端口
协议

进行转发。

缺点：

无法根据 URL、Header、Cookie 等应用层内容做精细路由。

考试中看到：

四层 LVS
传输层
高性能
IP/端口转发

基本就是这个点。

5.3 七层负载均衡

七层指应用层，常见于 HTTP/HTTPS。

典型技术：

Nginx
HAProxy
API 网关
Ingress Controller
云厂商 ALB

特点：

- 可以识别 HTTP 请求内容。
- 可以按域名、URL、Header、Cookie 路由。
- 可以做 HTTPS 终止。
- 可以做灰度发布、限流、鉴权、重写、缓存。

例如：

/login -> 登录服务
/pay -> 支付服务
/rank -> 排行榜服务
Header: gray=true -> 灰度版本
Cookie: userGroup=A -> A 版本

缺点：

处理应用层协议，开销通常高于四层负载均衡。

考试中看到：

七层 Nginx
应用层
URL 路由
反向代理
HTTP Header

基本就是这个点。

5.4 服务端负载均衡

客户端请求先到负载均衡器，由负载均衡器转发到后端。

结构：

客户端 -> 负载均衡器 -> 服务实例

典型：

- Nginx。
- LVS。

- 云 SLB。
- API 网关。

优点：

- 客户端简单。
- 统一入口。
- 便于集中治理。

缺点：

- 负载均衡器本身要做高可用。
- 如果设计不好，负载均衡器可能成为瓶颈。

5.5 客户端负载均衡

客户端或调用方自己从服务注册中心拿服务列表，然后选择一个服务实例调用。

结构：

服务消费者 -> 服务注册中心 -> 获取实例列表
服务消费者 -> 按策略选择服务实例 -> 调用

常见于微服务框架。

优点：

- 少一次集中代理转发。
- 调用方可感知服务实例列表。
- 适合内部服务调用。

缺点：

- 客户端逻辑复杂。
- 多语言 SDK 维护成本高。
- 策略升级需要考虑客户端版本。

5.6 数据层负载均衡

负载均衡不仅用于应用服务器，也用于数据库和缓存。

常见方式：

- 数据库读写分离。
- 主库写，从库读。
- 分库分表。
- 分片路由。
- 缓存集群分片。
- 一致性哈希。

例如：

写请求 -> 主库
读请求 -> 从库1 / 从库2 / 从库3
玩家ID hash -> 指定分库
排行榜ID hash -> 指定 Redis 分片

考试中经常把负载均衡和数据库读写分离、缓存、分布式数据库一起考。

6. 常用负载均衡策略

这一部分是你必须掌握的核心。

6.1 轮询 Round Robin

按顺序依次分配请求。

示例：

第1个请求 -> A
第2个请求 -> B
第3个请求 -> C
第4个请求 -> A
第5个请求 -> B

优点：

- 简单。
- 开销小。
- 实现容易。

缺点：

- 默认认为每台服务器能力一样。
- 默认认为每个请求处理成本差不多。
- 如果某些请求耗时长，会造成负载不均。

适合：

后端实例性能接近
请求处理成本接近
无状态服务

游戏例子：

公告查询、配置拉取、普通运营后台查询。

6.2 加权轮询 Weighted Round Robin

给不同服务器设置不同权重，权重越高，分配到的请求越多。

示例：

A 权重 5
B 权重 3

C 权重 2

大致分配比例：

A:B:C = 5:3:2

适合：

- 服务器配置不同。
- 新旧机器混合部署。
- 灰度发布时给新版本较小流量。
- 某个实例性能更强。

优点：

比轮询更灵活。

缺点：

权重通常需要人工配置或依赖监控动态调整。

游戏例子：

新版本活动服务先给 5% 流量，确认稳定后逐步提高权重。

6.3 随机 Random

随机选择一个后端节点。

优点：

- 实现简单。
- 在请求数量足够大时，整体分布相对均匀。

缺点：

- 短时间内可能不均匀。
- 不考虑服务器性能和实时负载。

适合：

请求量大、节点能力接近、对短期均衡要求不高的场景。

6.4 加权随机 Weighted Random

在随机基础上加入权重。

权重高的节点被选中的概率更高。

适合：

- 节点性能不同。
- 请求量较大。

- 希望实现简单但又能体现机器差异。

6.5 最少连接 Least Connections

把新请求分配给当前连接数最少的节点。

适合：

- 请求处理时间差异较大。
- 长连接场景。
- WebSocket。
- 游戏网关长连接。
- 下载、上传、流式处理。

优点：

比轮询更能反映实时压力。

缺点：

需要维护连接数统计。
连接数少不一定代表 CPU、内存和业务负载低。

游戏例子：

长连接网关可以参考连接数分配玩家连接。

6.6 加权最少连接

在最少连接基础上考虑服务器权重。

适合：

服务器性能不同，同时连接持续时间差异较大。

例如：

A 机器配置高，可以承载更多连接。
B 机器配置低，连接数阈值更低。

6.7 最短响应时间 Least Response Time

把请求分配给响应时间较短的节点。

它更接近真实用户体验。

适合：

- 请求耗时差异明显。
- 服务性能波动较大。
- 需要动态调整流量的场景。

优点：

能感知节点处理慢的问题。

缺点：

响应时间统计可能有延迟。
偶发慢请求可能导致误判。
需要和健康检查、错误率综合判断。

6.8 基于资源利用率

根据 CPU、内存、磁盘 I/O、网络带宽、队列积压等指标分配请求。

适合：

- 实例性能差异明显。
- 请求成本差异大。
- 有完善监控体系。

优点：

更动态、更贴近真实负载。

缺点：

实现复杂。
监控数据有采样周期和延迟。
指标不准会导致错误调度。

6.9 IP Hash

根据客户端 IP 计算哈希值，将同一个 IP 的请求固定分配到同一个后端节点。

优点：

- 可以实现简单的会话保持。
- 同一客户端请求落到同一节点。

缺点：

- NAT 场景下大量用户可能共享同一个出口 IP，导致热点。
- 节点增减时映射关系变化较大。
- 不适合强依赖精确用户身份的场景。

适合：

简单会话保持
缓存命中
轻量粘性路由

6.10 一致性哈希 Consistent Hash

一致性哈希常用于缓存、分片和有状态服务路由。

它解决的问题：

普通 hash 在节点增减时，大量 key 会重新映射；
一致性 hash 可以减少节点变化造成的数据迁移范围。

适合：

- 缓存分片。
- Redis 集群路由。
- 玩家 ID 路由。
- 房间 ID 路由。
- Actor 路由。
- 有状态服务。

游戏例子：

playerId -> hash -> 玩家状态服务实例
roomId -> hash -> 房间服务实例
activityId -> hash -> 活动分片

优点：

有利于状态一致性和缓存命中。
节点增减时迁移范围较小。

缺点：

可能出现热点 key。
需要虚拟节点或分片拆分缓解不均衡。

6.11 会话保持 Sticky Session

会话保持不是具体算法，而是一类要求：

同一个用户或同一个会话的请求尽量落到同一台后端服务器。

实现方式：

- IP Hash。
- Cookie。
- Session ID。
- Token 中的用户 ID。
- 业务 ID Hash。

适合：

- 后端保存本地会话。
- 长连接。
- 有状态业务。
- 缓存命中要求较高。

风险：

- 某个热点用户或热点业务可能压垮单个节点。
- 节点故障时会话丢失。
- 不利于无状态水平扩展。

考试建议：

能无状态就尽量无状态；
必须有状态时，再通过会话保持、业务分片或一致性哈希处理。

6.12 主备和故障转移

有些场景不是多个节点同时分流，而是主节点处理，备节点等待。

结构：

主节点工作
备节点待命
主节点故障 -> 切换到备节点

这更偏向可用和故障转移，不一定是典型负载均衡。

考试容易混淆：

负载均衡：多个节点分担流量。
故障转移：主节点故障时切换到备用节点。
负载分担：多条链路或节点同时分担流量。
冗余备份：备用资源用于避免单点故障。

7. 静态、动态和基于场景的负载均衡

论文和案例题很喜欢用这个分类。

7.1 静态负载均衡

按预设规则分发，不依赖实时运行状态。

常见策略：

轮询
加权轮询
随机
加权随机
哈希

优点：

- 简单。
- 性能开销低。
- 容易实现。

缺点：

- 不感知实时负载。
- 节点异常时需要配合健康检查。

- 请求成本差异大时容易不均衡。

适合：

无状态服务
节点性能接近
请求成本差异不大

7.2 动态负载均衡

根据节点实时状态调整流量。

参考指标：

健康状态
连接数
响应时间
CPU
内存
错误率
超时率
队列积压
磁盘 I/O
网络带宽

优点：

- 能避开异常节点。
- 能降低高负载节点压力。
- 更适合复杂业务。

缺点：

- 依赖监控数据。
- 实现复杂。
- 指标延迟或误判可能造成抖动。

适合：

请求耗时差异大
实例状态波动明显
需要自动摘除故障节点

7.3 基于场景的负载均衡

这才是架构师视角。

不同业务不能简单套同一个算法。

场景	推荐思路
登录服务	无状态优先，轮询/加权轮询 + 健康检查 + 限流
配置拉取	CDN/缓存 + 轮询
长连接网关	最少连接 + 会话保持
玩家状态服务	按 playerId 哈希或 Actor 路由
房间/匹配服务	按 roomId/teamId 路由，避免状态冲突
排行榜	缓存分片 + 热点隔离
活动服务	提前扩容 + 分区 + 限流降级

支付回调	独立入口 + 保守策略 + 幂等和审计
数据库读请求	读写分离，读从库
缓存访问	一致性哈希或集群分片

关键记忆：

负载均衡要看服务是否有状态、请求成本是否均衡、是否有热点、是否要求强一致。

8. 四层 LVS 和七层 Nginx 怎么选

软考案例中常出现：

四层 LVS
七层 Nginx

8.1 LVS 适合什么

LVS 工作在四层，基于 IP 和端口做转发。

适合：

- 高并发。
- 高性能转发。
- TCP/UDP 流量。
- 不需要识别 HTTP URL 的场景。
- 入口层粗粒度流量分发。

优点：

性能高，转发效率高。

缺点：

应用层路由能力弱。

8.2 Nginx 适合什么

Nginx 常作为七层反向代理和负载均衡。

适合：

- HTTP/HTTPS。
- 按域名路由。
- 按 URL 路由。
- 灰度发布。
- 静态资源。
- 反向代理。
- HTTPS 终止。
- Header/Cookie 路由。

优点：

应用层能力强，配置灵活。

缺点：

应用层处理开销通常高于四层转发。

8.3 常见组合

大型系统常组合使用：

DNS/GSLB -> LVS/云四层负载均衡 -> Nginx/API 网关 -> 应用服务集群

理解：

前面负责大流量接入和高性能转发；

后面负责应用层路由、鉴权、限流、灰度和服务治理。

9. 反向代理、CDN 和负载均衡的关系

9.1 反向代理

反向代理位于客户端和后端服务器之间，对客户端表现为服务端。

它可以做：

- 请求转发。
- 缓存。
- SSL 终止。
- 压缩。
- 限流。
- 负载均衡。
- 隐藏后端服务。

Nginx 就经常作为反向代理。

9.2 CDN

CDN 把静态资源或可缓存内容放到离用户更近的节点。

作用：

- 加快访问速度。
- 降低源站压力。
- 缓解地域网络差异。

适合：

- 图片。
- 安装包。
- 游戏资源包。

- 热更新资源。
- 静态配置。
- 视频。

教材重点：

CDN 和反向代理的基本原理都是缓存。
二者目的都是尽早返回数据给用户，同时减轻后端服务器负载压力。

9.3 三者关系

CDN：把内容前置到离用户近的地方，减少回源。
反向代理：站点入口代理后端服务，可缓存、转发、隐藏后端。
负载均衡：在多个后端节点之间分配请求。

它们可以一起用：

玩家 -> CDN -> 反向代理/Nginx -> 负载均衡 -> 游戏服务实例

10. 健康检查和故障摘除

负载均衡如果只分流，不检查后端是否健康，会把请求转发给坏节点。

健康检查通常分为：

10.1 TCP 检查

检查端口是否能建立连接。

优点：

简单、快速。

缺点：

端口通不代表业务正常。

10.2 HTTP 检查

请求后端健康检查接口，例如：

GET /health

如果返回 200，则认为健康。

优点：

能检查应用层状态。

缺点：

健康检查接口设计不合理会误判。

10.3 业务健康检查

进一步检查依赖项。

例如：

- 数据库连接是否正常。
- Redis 是否正常。
- 队列是否积压严重。
- 关键配置是否加载成功。

注意：

健康检查不能太重，否则健康检查本身会增加系统压力。

10.4 故障摘除

当节点异常时，负载均衡器应停止向它分发请求。

恢复后，可以重新加入。

常用机制：

- 连续失败 N 次摘除。
- 连续成功 M 次恢复。
- 慢启动逐步增加权重。
- 熔断一段时间后探测恢复。

考试表达：

通过健康检查及时发现异常节点，并将其从负载均衡池中摘除；
待节点恢复后再逐步加入，避免故障节点持续接收流量。

11. 负载均衡和有状态服务

这是容易出错的地方。

11.1 无状态服务最适合负载均衡

无状态服务指：

任意请求发到任意实例都能正确处理。

例如：

- 配置查询。
- 公告查询。
- 普通列表查询。
- 幂等接口。

无状态服务可以简单扩容：

增加实例 -> 自动分担流量

11.2 有状态服务不能随便分

有状态服务指：

请求处理依赖某个节点上的本地状态。

例如：

- 玩家在线状态。
- 房间状态。
- 匹配队列。
- 战斗状态。
- 本地 Session。

如果随机转发，可能出现：

- 状态找不到。
- 并发冲突。
- 数据不一致。
- 会话丢失。
- 多节点重复处理。

解决方式：

会话保持
业务 ID Hash
一致性哈希
Actor 路由
状态外置到 Redis/数据库
服务无状态化

11.3 推荐思路

架构师要优先考虑：

能无状态化就无状态化；
不能无状态化，就按业务键稳定路由；
热点明显时，再做分片、拆分、迁移和限流。

对你熟悉的 Orleans 来说：

玩家 Grain、房间 Grain、匹配 Grain 的路由，本质上就是一种基于业务身份的负载分配和状态定位。
它不能简单随机，否则状态一致性会出问题。

12. 负载均衡和数据库

应用层负载均衡解决不了数据库瓶颈。

大型网站演进中，应用服务器集群之后，数据库常成为瓶颈，所以会出现：

数据库读写分离
主从复制
分库分表
分布式数据库
缓存

搜索引擎

12.1 读写分离

写请求 -> 主库
读请求 -> 从库

作用：

- 分担读压力。
- 提升查询能力。
- 降低主库负载。

风险：

- 主从延迟。
- 读到旧数据。
- 一致性处理复杂。

12.2 分库分表

按业务或数据规则拆分。

例如：

按游戏区服分库
按玩家 ID 分表
按订单时间分表

作用：

- 降低单库单表压力。
- 分散存储和查询负载。

风险：

- 跨库查询复杂。
- 分布式事务复杂。
- 扩容迁移复杂。

12.3 缓存分片

Redis、Memcached 等缓存也要负载均衡。

常用：

一致性哈希
Redis Cluster
业务分片
热点 key 拆分

13. 负载均衡和高可用

负载均衡是高可用架构的重要组成部分，但不是高可用的全部。

完整高可用通常还需要：

- 多实例部署。
- 健康检查。
- 故障摘除。
- 自动恢复。
- 限流。
- 熔断。
- 降级。
- 缓存。
- 消息队列削峰。
- 数据库主从。
- 多机房容灾。
- 监控告警。

考试表达：

负载均衡通过多实例分发请求避免单点故障，但仍需要结合健康检查、故障转移、限流降级、监控告警和数据层高可用，才能形成完整的高可用体系。

14. 用你的游戏项目理解负载均衡

你是手机游戏开发工程师，可以按游戏链路理解。

14.1 登录高峰

问题：

开服、版本更新、活动开启时，大量玩家同时登录。

策略：

- 登录服务多实例。
- 接入层负载均衡。
- 健康检查。
- 动态扩容。
- 限流和排队。
- Redis 缓存账号基础信息。

适合算法：

无状态登录接口：轮询、加权轮询、最短响应时间。

14.2 游戏长连接网关

问题：

玩家连接保持时间长，不同网关连接数可能差异很大。

策略：

- 按最少连接分配。
- 按大区或玩家 ID 分片。
- 网关健康检查。
- 断线重连时尽量回到原网关或能恢复会话。

适合算法：

最少连接、加权最少连接、会话保持。

14.3 玩家状态服务

问题：

玩家状态不能随机落到不同节点，否则会出现并发修改和状态不一致。

策略：

- 按 playerId 路由。
- 使用 Orleans Grain 定位玩家状态。
- 状态持久化。
- 热点玩家或热点活动单独处理。

适合算法：

业务 ID Hash、一致性哈希、Actor 路由。

14.4 排行榜

问题：

排行榜查询和刷新会形成热点。

策略：

- 排行榜缓存。
- 按榜单 ID 分片。
- 热门榜单单独扩容。
- 降低非核心榜单刷新频率。
- 读写分离。

适合算法：

一致性哈希、缓存分片、热点隔离。

14.5 支付回调

问题：

支付回调必须可靠、可追踪、可靠等。

策略：

- 独立支付回调入口。
- 多实例部署。
- 健康检查。
- 幂等处理。
- 回调日志和审计。
- 失败重试。

注意：

支付链路不能只追求均衡，还要优先保证幂等性、一致性和可追踪性。

15. 选择题常见考点

15.1 定义

负载均衡是把请求或任务分配到多个处理节点，避免单点过载，提高性能、可用性和扩展性。

15.2 大型网站演进

常考顺序：

单体
应用和数据分离
缓存
服务集群 + 负载均衡
数据库读写分离
反向代理 + CDN
分布式文件系统和分布式数据库
NoSQL 和搜索引擎
业务拆分
分布式服务

负载均衡对应：

使用服务集群改善网站并发处理能力。

15.3 四层和七层

对比项	四层负载均衡	七层负载均衡
所在层	传输层	应用层
依据	IP、端口、TCP/UDP	URL、Header、Cookie、HTTP 内容
典型技术	LVS	Nginx、HAProxy、API 网关
优点	性能高	路由灵活
缺点	应用层能力弱	开销相对高

常见判断：

LVS 更偏四层。
Nginx 更常用于七层反向代理和负载均衡。

15.4 静态和动态策略

静态：轮询、加权轮询、随机、哈希。

动态：最少连接、最短响应时间、基于健康状态、CPU、错误率、队列积压。

15.5 健康检查

负载均衡要结合健康检查。

否则：

坏节点仍会接收请求。

15.6 会话保持

会话保持用于有状态服务。

但考试要注意：

会话保持会降低调度灵活性，可能造成热点；

更推荐将服务设计为无状态，或将状态外置。

15.7 负载均衡和冗余

不要混淆：

负载均衡：多个节点共同分担请求。

主备切换：主节点失败后切换到备节点。

负载分担：多条链路或多个节点同时承担流量。

冗余设计：通过备用资源避免单点故障。

冲刺资料中有类似考点：

备用路径通常是在主路径失效时启用；

负载分担是通过并行链路分担流量来提高性能。

15.8 容器编排

Kubernetes 相关考点：

容器编排能力包括资源调度、应用部署与管理、自动修复、服务发现与负载均衡、弹性伸缩等。

15.9 Serverless

Serverless 相关考点：

资源调度服务需要负载均衡和流控。

通过分片和横向扩展避免单点瓶颈。

15.10 数据库性能指标

冲刺资料中提到：

衡量数据库管理系统的主要性能指标包括最大并发事务处理能力、负载均衡能力、最大连接数等。

16. 案例题常见问法和答题模板

16.1 问：如何提高系统并发处理能力

答题模板：

可以将应用服务部署为集群，在接入层引入负载均衡器，将用户请求分发到多个应用实例。
负载均衡器根据轮询、加权轮询、最少连接或响应时间等策略选择后端节点，
并通过健康检查及时摘除异常实例。
同时结合缓存、数据库读写分离、消息队列削峰、限流降级和横向扩容，
避免单点服务器成为系统瓶颈。

16.2 问：四层 LVS 和七层 Nginx 如何选择

答题模板：

LVS 工作在四层，主要依据 IP、端口和 TCP/UDP 连接进行转发，性能高，适合高并发入口流量分发。
Nginx 常用于七层负载均衡和反向代理，可以根据域名、URL、Header、Cookie 等应用层信息路由，
适合 HTTP 服务、灰度发布、反向代理、静态资源和应用层治理。
实际系统中可以组合使用，前端用四层负载均衡承接大流量，后端用七层网关做精细路由和治理。

16.3 问：如何处理有状态服务的负载均衡

答题模板：

有状态服务不能简单随机分发请求，否则可能导致会话丢失、状态冲突或数据不一致。
应优先考虑服务无状态化，将状态外置到数据库、缓存或共享存储中。
如果必须保留本地状态，可以采用会话保持、业务 ID 哈希、一致性哈希或 Actor 路由，
保证同一用户、房间或业务对象的请求稳定路由到同一处理节点。
同时要监控热点对象，必要时进行分片、迁移、限流或单独扩容。

16.4 问：负载均衡如何提升可用性

答题模板：

负载均衡通过多实例部署避免单点故障。
负载均衡器对后端实例进行健康检查，当实例异常、响应超时或错误率升高时，
将其从服务池中摘除，避免请求继续转发到故障节点。
实例恢复后可以逐步加入服务池。
结合多副本、自动扩容、限流降级、监控告警和数据层主从复制，
可以提升系统整体可用性和故障恢复能力。

16.5 问：如何设计电商秒杀或游戏活动高峰

答题模板：

活动高峰具有瞬时流量大、热点访问集中和响应时间要求高的特点。
可以在接入层采用负载均衡和限流，将请求分发到多个活动服务实例；
活动开始前进行容量评估、预扩容和缓存预热；
对配置、商品、排行榜等热点数据使用 CDN、缓存和分片；
对非核心操作采用消息队列异步处理；
对异常实例通过健康检查及时摘除；
对支付、发奖等关键链路采用幂等、补偿和审计机制。

17. 论文可用表达

如果论文题目是“论负载均衡技术”“论系统负载均衡设计方法”“论高并发系统架构设计”，可以写：

在项目中，随着玩家规模扩大和运营活动增多，系统在开服、版本更新、限时活动和排行榜刷新时
出现瞬时流量高、服务实例负载不均和单点故障影响用户请求等问题。
为此，我们在接入层、服务层和数据层分别设计负载均衡方案。

接入层通过网关和负载均衡器将请求分发到多个服务实例，并结合健康检查、故障摘除和动态权重调整。
对于登录、公告和配置拉取等无状态接口，采用轮询和加权轮询；
对于长连接网关，采用最少连接策略；
对于玩家状态、房间和匹配等有状态业务，采用基于玩家 ID 或房间 ID 的一致性哈希和 Actor 路由；
对于排行榜、活动等热点数据，采用缓存分片、读写分离和热点隔离。
同时建立 QPS、响应时间、错误率、CPU、队列积压等监控指标，
在活动高峰前进行容量评估和预扩容，在异常时通过限流、降级和故障摘除保障核心链路稳定。

18. 快速记忆

18.1 一句话

负载均衡 = 把请求合理分到多个节点，避免单点过载，提高性能、可用性和扩展性。

18.2 三类策略

静态：轮询、加权轮询、随机、哈希。

动态：最少连接、响应时间、健康状态、资源利用率。

场景：无状态用轮询，有状态用哈希，长连接看连接数，热点用缓存和分片。

18.3 四层七层

四层看 IP 和端口，LVS 性能高。

七层看 URL 和 Header，Nginx 更灵活。

18.4 健康检查

发现坏节点，摘除坏节点，恢复后再加入。

18.5 有状态服务

不要随机分，按业务 ID 稳定路由。

18.6 负载均衡不是万能

应用层均衡解决不了所有问题；

数据库、缓存、队列、限流、熔断、监控都要配合。

19. 考前最低掌握清单

如果时间紧，至少背下面这些：

1. 负载均衡用于将请求或任务分配到多个节点，避免单点过载，提高性能、可用性和扩展性。
2. 大型网站架构中，服务集群通过负载均衡调度服务器改善并发处理能力。
3. 常见静态策略：轮询、加权轮询、随机、哈希。
4. 常见动态策略：最少连接、最短响应时间、健康状态、资源利用率。
5. 四层负载均衡依据 IP、端口、TCP/UDP，典型技术 LVS，性能高。
6. 七层负载均衡依据 URL、Header、Cookie 等应用层信息，典型技术 Nginx，路由灵活。
7. 健康检查和故障摘除是负载均衡高可用的关键。
8. 无状态服务适合轮询和加权轮询，有状态服务适合会话保持、业务 ID 哈希或一致性哈希。
9. CDN 和反向代理主要通过缓存加快访问并减轻后端压力。
10. 负载均衡要结合缓存、读写分离、消息队列、限流降级、监控告警一起设计。

20. 资料依据

- 本地教材：最新教材-新大纲/03、【带搜索】系统架构设计师第二版.pdf，第 10

章“大型网站系统架构演化实例”中服务集群、负载均衡调度服务器、反向代理、CDN、数据库读写分离等内容。

- 本地教材：最新教材-新大纲/03、【带搜索】系统架构设计师第二版.pdf，第 14 章“云原生架构设计理论与实践”中 Kubernetes 服务发现与负载均衡、微服务负载均衡、Serverless 负载均衡和流控相关内容。
- 本地冲刺资料：最新教材-新大纲/04、考试32小时通关-第2版（新版）.pdf，第 12 小时大型网站架构演进、第 17 小时云原生架构、模拟案例中四层 LVS 与七层 Nginx 的负载均衡场景。
- 本地论文练习资料：复习规划/论文练习/2025上半年_论负载均衡技术_参考论文_v2.md、复习规划/论文练习/2025上半年_论系统负载均衡设计方法_参考论文.md。